

Multimodal Deep Learning

Solutions to Exercise Sheet 7

Date: 8th of July, 2025

Exercise 1: Selective Parameter Fine-Tuning

- (a) **Fishmask** A layer contains 4.0×10^6 weights. A one-shot Fisher score approximation assigns each weight a scalar importance I_i . You may update at most **0.8%** of the weights.

- (i) Write the formula that maps the vector of scores $\mathbf{I}$ to a binary mask $\mathbf{m} \in \{0, 1\}^n$ respecting the budget constraint.
- (ii) To update the selected weights, the Adam optimizer requires memory for more than just the weights themselves. For each trainable parameter, memory must be allocated for its **gradient**, its **1st moment**, and its **2nd moment**. Assuming all values are stored in 32-bit float precision (4 bytes per number), what is the total memory overhead for the optimizer?

- (b) **DiffPruning**

You fine-tune a 350 M-parameter model, targeting a sparsity of **99.5%**.

- (i) Compute the GPU memory overhead (in MB) of storing the *dense* differentiable mask during training.
- (ii) If you could store the final mask as a packed 1-bit tensor for inference, what would its size be (in kB)?

- (c) **BitFit**

A GPT-style model has 40 transformer blocks with the following per-block sizes:

- Query bias: $d_{\text{model}} = 768$
- Middle-MLP bias: $d_{\text{ff}} = 3072$

- (i) Calculate the *percentage* of the model's 250 M parameters that BitFit updates.
- (ii) State one reason why BitFit's advantage fades on very large models or on architectures like LLaMA.

Solution

- (a) (i) $m_i = \mathbf{1}[I_i \geq \tau]$ where τ is the $(1 - \text{budget})$ -quantile of $\mathbf{I}$.
- (ii) Trainable weights = $0.008 \times 4.0 \times 10^6 = 32\,000$.
 Optimizer overhead = $\underbrace{32\,000}_{\text{params}} \times \underbrace{3}_{\text{grad+moments}} \times \underbrace{4\text{ B}}_{\text{float32}} = 384\,000\text{ B} = \mathbf{384\text{ KB}}$.

Pitfall to recall: Computing $\mathbf{I}$ itself may dominate wall-clock time because it requires second-order statistics.

- (b) (i) Mask elements = 350 M. Memory = $350 \times 10^6 \times 4\text{ B} \approx \mathbf{1.34\text{ GB}}$.

- (ii) Packed size = 350×10^6 bits = 43.75 MB = 44 MB. At inference you still need indices for the 0.5% previously finetuned weights; using 32-bit indices that is 1.75 MB, so total $\approx$ **46 MB**.

Although update-sparse, DiffPruning is memory-heavy during optimisation.

- (c) (i) Tunable per block = $768 + 3072 = 3840$. For 40 blocks: $3840 \times 40 = 153\,600$ parameters. Percentage = $\frac{153\,600}{250 \times 10^6} \times 100 \approx$ **0.061 %**.
 (ii) Bias terms only have a limited expressivity in large neural networks. Additional capacity lives in weight matrices. Moreover, some modern architectures remove biases entirely, leaving BitFit with nothing to tune.

Exercise 2: Adapters

- (a) You are adding adapter modules to a transformer with a hidden dimension of $d_{\text{model}} = 768$. Each adapter is a bottleneck feed-forward network with a reduction factor, such that the inner dimension is $r = 16$. The adapter structure is:

$$\text{Linear}(d_{\text{model}} \rightarrow r) \rightarrow \text{GELU} \rightarrow \text{Linear}(r \rightarrow d_{\text{model}})$$

- (i) Calculate the number of trainable parameters in a **single** adapter module. Remember to include bias terms for both linear layers.
 (ii) Explain why this method, despite its parameter efficiency, significantly increases inference latency compared to a method like (IA)³.

(b) **(IA)³**

You are fine-tuning the same 40-block, 250 M-parameter model from the BitFit exercise using (IA)³. This method introduces three learnable scaling vectors per transformer block:

- $\mathbf{l}_v$ and $\mathbf{l}_k$ to scale the value and key matrices (each of size $d_{\text{model}} = 768$).
- $\mathbf{l}_{\text{ffn}}$ to scale the output of the first MLP layer (of size $d_{\text{ff}} = 3072$).

- (i) Calculate the total number of trainable parameters introduced by (IA)³ for the entire 40-block model.
 (ii) Based on its mechanism, why is (IA)³ considered to have more **limited expressibility** than a standard adapter module?

Solutions

- (a) (i) The total parameters are the sum of weights and biases for the two linear layers:
- Down-projection: $(d_{\text{model}} \times r) + r = (768 \times 16) + 16 = 12288 + 16 = 12304$
 - Up-projection: $(r \times d_{\text{model}}) + d_{\text{model}} = (16 \times 768) + 768 = 12288 + 768 = 13056$
- Total parameters per module = $12304 + 13056 =$ **25360**.
- (ii) Adapters introduce **new, sequential operations**. Each time an input passes through the model, it must be processed by these additional adapter layers. This adds computational steps and data movement, increasing the wall-clock time required for a forward pass. This is even exacerbated in multi-GPU training scenarios due to the need for repeated broadcasting. In contrast, methods like (IA)³ involve scaling operations that can often be **fused** into existing computations at inference time, adding negligible latency.

- (b) (i) Parameters per block = $\underbrace{768}_{l_v} + \underbrace{768}_{l_k} + \underbrace{3072}_{l_{ffn}} = 4608$.
 Total parameters for 40 blocks = $4608 \times 40 = 184320$.
 This is $\frac{184320}{250 \times 10^6} \times 100 \approx \mathbf{0.074\%}$ of the model's parameters.
- (ii) (IA)³ can only perform element-wise **scaling** of existing activation vectors. It can make certain features more or less prominent, but it cannot perform complex, non-linear transformations or rotations in the activation space. A standard adapter, containing an MLP with a non-linearity, can learn a much richer family of functions to modify the activations, giving it greater expressive power.

Exercise 3: Multi-Modal Fusion Strategies

- (a) **Representation Fusion** You are predicting house prices using two modalities: an image (A) and a text description (B). After passing through individual encoders, you have the feature vectors:

$$\mathbf{x}_A = \begin{bmatrix} 2 \\ 8 \\ 1 \\ 3 \end{bmatrix} \in \mathbb{R}^4 \quad \text{and} \quad \mathbf{x}_B = \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix} \in \mathbb{R}^3$$

1. **Concatenation Fusion:** Compute the joint representation $\mathbf{z}_{\text{concat}}$ and state its dimension.
 2. **Additive Fusion:**
 - (a) Can you directly apply additive fusion ($\mathbf{z} = \mathbf{x}_A + \mathbf{x}_B$)? Why or why not?
 - (b) To enable additive fusion, you must project both vectors into a common dimension $d_z = 2$ using matrices $\mathbf{W}_A \in \mathbb{R}^{2 \times 4}$ and $\mathbf{W}_B \in \mathbb{R}^{2 \times 3}$. Write the complete formula for the fused representation $\mathbf{z}_{\text{add}}$.
 3. **Bilinear Fusion:**
 - (a) Compute the joint representation matrix $\mathbf{Z}_{\text{bilinear}}$ using the outer product ($\mathbf{Z} = \mathbf{x}_A \mathbf{x}_B^T$).
 - (b) What is the shape of $\mathbf{Z}_{\text{bilinear}}$? How do its rows and columns relate to the input modalities?
 4. Why is Bilinear Fusion not directly applicable for combining three or more modalities simultaneously? What more general fusion method overcomes this?
- (b) **Prediction Fusion** A content moderation system uses three separate models (Text, Image, Audio) to detect hate speech. For one post, they output the following probabilities of hate speech:

$$p_{\text{text}} = 0.90 \quad p_{\text{image}} = 0.60 \quad p_{\text{audio}} = 0.15$$

1. *Simple rules.*
 - (a) Using plain averaging and threshold 0.5, is the post flagged?
 - (b) Apply majority *voting* with a 0.7 modality-specific threshold (1 = hate, 0 = non-hate). Give the final binary decision.
2. *Weighted averaging.* Suppose validation AUROC scores are 0.93 (text), 0.78 (image) and 0.70 (audio).
 - (a) Propose weights proportional to AUROC, normalised to sum 1.

- (b) Compute the weighted probability and state the binary outcome at threshold 0.5.
- (c) Discuss one advantage of weighting over unweighted averaging.
- 3. *Probabilistic fusion.* Under an “AND” logic (multiply probabilities) calculate the joint probability and comment on when such a rule may be preferable despite lower recall.
- 4. *Model-based post-processing.* Outline how you would prompt an LLM to reconcile disagreeing modalities. Specify what information you would feed (e.g. class labels, confidence scores, textual rationale snippets).

Solution

- (a) Given $\mathbf{x}_A = [2, 8, 1, 3]^T \in \mathbb{R}^4$ and $\mathbf{x}_B = [4, 1, 5]^T \in \mathbb{R}^3$.

1. **Concatenation Fusion:** The joint representation is formed by stacking the vectors.

$$\mathbf{z}_{\text{concat}} = [\mathbf{x}_A^T, \mathbf{x}_B^T]^T = [2, 8, 1, 3, 4, 1, 5]^T$$

The resulting dimension is $d_z = d_A + d_B = 4 + 3 = 7$. Thus, $\mathbf{z}_{\text{concat}} \in \mathbb{R}^7$.

2. **Additive Fusion:**

- (a) No, direct additive fusion is not possible. It is an element-wise operation that requires vectors to have the same dimension. Here, $\mathbf{x}_A \in \mathbb{R}^4$ and $\mathbf{x}_B \in \mathbb{R}^3$ have different dimensions.
- (b) To enable addition, we first project each vector into the common space $\mathbb{R}^2$. The complete formula for the fused representation is the sum of these projected vectors:

$$\mathbf{z}_{\text{add}} = \mathbf{W}_A \mathbf{x}_A + \mathbf{W}_B \mathbf{x}_B$$

3. **Bilinear Fusion:**

- (a) The joint representation is the outer product $\mathbf{Z}_{\text{bilinear}} = \mathbf{x}_A \mathbf{x}_B^T$.

$$\mathbf{Z}_{\text{bilinear}} = \begin{bmatrix} 2 \\ 8 \\ 1 \\ 3 \end{bmatrix} \begin{bmatrix} 4 & 1 & 5 \end{bmatrix} = \begin{bmatrix} 8 & 2 & 10 \\ 32 & 8 & 40 \\ 4 & 1 & 5 \\ 12 & 3 & 15 \end{bmatrix}$$

- (b) The shape of $\mathbf{Z}_{\text{bilinear}}$ is $d_A \times d_B$, which is 4×3 . Its rows correspond to the feature dimensions of modality A (image), and its columns correspond to the features of modality B (text). Each element Z_{ij} captures the pairwise multiplicative interaction between the i -th feature of $\mathbf{x}_A$ and the j -th feature of $\mathbf{x}_B$.
 - 4. Bilinear fusion is defined as the outer product of exactly two vectors, yielding a rank-2 tensor (a matrix). It cannot be applied to three or more modalities simultaneously. The more general method that overcomes this is **Tensor Fusion**, which computes the tensor product of any number of modality vectors, resulting in a higher-order tensor.
- (b) Given probabilities $p_{\text{text}} = 0.90$, $p_{\text{image}} = 0.60$, and $p_{\text{audio}} = 0.15$.

1. *Simple rules.*

- (a) **Averaging:** The average probability is $(0.90 + 0.60 + 0.15)/3 = 0.55$. Since $0.55 > 0.5$, the post is **flagged**.
- (b) **Voting:** Applying a 0.7 threshold gives binary votes: Text ($0.90 > 0.7 \implies 1$), Image ($0.60 < 0.7 \implies 0$), Audio ($0.15 < 0.7 \implies 0$). The majority vote is 0, so the final decision is **non-hate**.

2. *Weighted averaging.*

- (a) Weights proportional to AUROC scores (0.93, 0.78, 0.70) are normalized by their sum (2.41): $w_{\text{text}} \approx 0.386$, $w_{\text{image}} \approx 0.324$, $w_{\text{audio}} \approx 0.290$.
- (b) The weighted probability is $(0.386 \times 0.90) + (0.324 \times 0.60) + (0.290 \times 0.15) \approx 0.585$. Since $0.585 > 0.5$, the post is **flagged**.
- (c) Advantage: It gives more influence to more reliable or performant modalities, leading to a more robust final decision than treating all models as equally important.

3. *Probabilistic fusion.* Using an "AND" logic (multiplication):

$$p_{\text{and}} = 0.90 \times 0.60 \times 0.15 = 0.081$$

This rule is preferable in high-precision scenarios where false positives are very costly, as it requires consensus from all modalities to issue a positive prediction. This increases precision at the cost of recall.

4. *Model-based post-processing.* An LLM can be prompted to act as an arbiter to reconcile disagreements. The prompt should provide the LLM with structured information, including:

- The prediction from each modality (e.g., "HATE", "NOT HATE").
- The confidence score for each prediction (e.g., 0.90).
- A concise, human-readable rationale snippet from each model explaining its decision (e.g., "Text contains a known slur," "Image contains an ambiguous symbol").

This allows the LLM to weigh the evidence from conflicting sources and make a more nuanced final judgment.

Exercise 4: Exercise: Multimodal Fusion for Radiology Reporting

Scenario A system receives three inputs for a single patient:

1. **Chest X-ray:** A single high-resolution 2D radiograph (e.g., 1024×1024 pixels).
2. **Patient meta-data:** Semi-structured information (e.g., Age: 65; Sex: M;).
3. **Referrer note:** Unstructured free text from the referring physician.

The goal is to autoregressively generate a structured radiology report, such as:

Findings: ... **Impression:** ...

Your task is to design a complete transformer-based pipeline for this problem. You will define the tokenization, choose an architecture, propose a training objective, and analyze its computational efficiency.

(a) Input Tokenisation and Representation

- i. **X-Ray Image Tokens** The raw image must be converted into a sequence of feature vectors. Outline one *soft token* strategy and one *hard token* strategy.
- ii. **Pros and Cons** For this specific radiology task, summarize two key pros and cons of your proposed soft vs. hard tokenization strategies. (Hint: Think about reconstruction fidelity, semantic abstraction, and the granularity of visual information).

- iii. **Text and Metadata Tokens** Propose a unified tokenization strategy for both the patient meta-data and the referrer note. How would you ensure the model can distinguish between these two types of textual information in the joint input sequence?
- iv. **Input Sequence Sketch** Sketch the final concatenated input sequence for your model. Clearly mark the boundaries between different modalities and indicate the embedding process for each.

(b) **Multimodal Architecture Design**

After tokenizing all inputs into a single sequence, you must choose an architecture to process it and generate the report. Pick **one** of the two architectural paradigms below and justify your selection based on its expected performance, training efficiency, and potential trade-offs.

- **(A) Standard Encoder-Decoder Transformer:**
 - A deep **Encoder** processes the complete multimodal input sequence via self-attention. Every token attends to every other token.
 - A **Decoder** then autoregressively generates the report tokens, using cross-attention to access the encoder’s full output at every step.
- **(B) Latent-Query Transformer (Perceiver-style):**
 - An initial **Cross-Attention** layer compresses the long multimodal input sequence into a short, fixed-size array of N learned latent vectors (queries).
 - A deep **Latent Transformer** (processing tower) refines these N vectors using self-attention, decoupled from the input length.
 - A **Decoder** autoregressively generates the report tokens, using cross-attention to access the final N latent vectors.

Explain how the standard training objective for generation: next-token prediction (causal language modeling loss), would be applied to your chosen architecture.

(c) **Training Strategy**

Training your chosen architecture from scratch on a limited number of full radiology reports is unlikely to succeed. A crucial step is large-scale pre-training on more general, unlabeled data to learn fundamental multimodal relationships.

Propose **one** self-supervised pre-training objective to teach your model how to connect vision and text. Describe the objective, the type of data it would require, and why it would be beneficial for the final radiology reporting task. (Hint: Consider objectives discussed in last week’s lecture).

(d) **Computational Analysis** Analyze the computational cost of the most expensive component in each architecture during a single forward pass. Assume:

- The input image produces $M_{img} = 4096$ tokens (from 16×16 patches). Text and metadata add $M_{text} = 256$ tokens. Total input tokens $M = 4352$.
- The hidden dimension is $d = 768$.
- For architecture (B), the latent array has a length of $N = 256$, and the processing tower has $L = 12$ layers.
- The approximate FLOPs for an attention layer are:
 - Self-Attention on a sequence of length S : $FLOPs \approx 2 \cdot S^2 \cdot d$
 - Cross-Attention with query length N and key/value length M : $FLOPs \approx 2 \cdot N \cdot M \cdot d$

- i. **Cost of Architecture A** The bottleneck in the Standard Encoder-Decoder is the encoder's self-attention. Calculate the FLOPs for **one** self-attention layer in the encoder, which processes the full sequence of M tokens.
- ii. **Cost of Architecture B** The Latent-Query model has two main computational steps: (i) the initial cross-attention layer to compress the M input tokens into N latents, and (ii) the L self-attention layers on the N latents. Calculate the total FLOPs for these components.
- iii. **Comparison** Which design is computationally cheaper, and by what approximate factor?
- iv. **Break-Even Point** Derive the equation for the input length M_{eq} at which the computational cost of one self-attention layer in Architecture A's encoder is equal to the cost of the initial cross-attention layer in Architecture B. Solve for M_{eq} using the given parameters. What does this value tell you?

Solution

(a) Input Tokenisation and Representation

i. X-Ray Image Tokens

- **Soft Token Strategy (Patch Projection):** The 1024×1024 image is divided into a grid of non-overlapping patches (e.g., 16×16 pixels). Each patch is flattened into a vector and passed through a learned linear projection to create a sequence of continuous feature vectors (soft tokens) of dimension d . This method is lossless at the input stage.
- **Hard Token Strategy (Vector Quantization):** A pre-trained visual tokenizer (e.g., VQ-GAN) encodes the image into a lower-resolution grid of latent vectors. Each vector is then replaced by the integer ID of its nearest neighbor in a finite, learned codebook. This creates a sequence of discrete "visual words."

ii. Pros and Cons for Radiology

• Soft Tokens (Patch Projection):

Pro: **Maximum Fidelity.** Preserves all subtle visual information (e.g., faint nodules, fine textures), which is critical for diagnostic accuracy.

Con: **No Semantic Prior.** The model must learn visual concepts from raw pixel patches, which is data-intensive and computationally demanding.

• Hard Tokens (Vector Quantization):

Pro: **Semantic Compression.** The codebook can learn to represent meaningful radiological concepts, simplifying the downstream generation task.

Con: **Information Loss.** The quantization step is lossy and may discard crucial diagnostic details not captured by the pre-trained tokenizer. The system's quality is bottlenecked by the tokenizer.

- iii. **Text and Metadata Tokens** A single Byte-Pair Encoding (BPE) tokenizer is used for both inputs. The metadata is first serialized into a string (e.g., "Age: 65; Sex: M;"). To differentiate the sources, special tokens are added to frame the content.
- iv. **Input Sequence Sketch** The final input is a single sequence of vectors, all of dimension d . Image patches are projected, while text/meta tokens are passed through an embedding lookup table.

$$\underbrace{[\text{img}_1] \dots [\text{img}_{M_{\text{img}}]}}_{\text{Image Tokens}} \underbrace{[\text{META}] \dots [\text{END}]}_{\text{Metadata Tokens}} \underbrace{[\text{NOTE}] \dots [\text{END}]}_{\text{Note Tokens}}$$

(b) **Multimodal Architecture Design****Selected Architecture: (B) Latent-Query Transformer (Perceiver-style)**

Rationale: The primary bottleneck in this problem is the long input sequence length ($M \approx 4k$) dominated by image tokens.

1. **Computational Efficiency:** A standard encoder's self-attention has a cost of $O(M^2)$, which is computationally prohibitive. The Latent-Query model replaces this with an initial cross-attention layer of cost $O(N \cdot M)$, which is linear in the long input length M . The subsequent processing happens in a deep but narrow latent space, with a cost of $O(L \cdot N^2)$, which is independent of the input length.
2. **Information Bottleneck:** This efficiency comes at the cost of an information bottleneck, as all information from the M input tokens must be compressed into N latent vectors. However, with a sufficiently large N (e.g., 256 or 512), this has been shown to be a highly effective trade-off.

Training Objective Application: The model is trained end-to-end with a standard **Causal Language Modeling** (next-token prediction) objective. The decoder autoregressively generates the report token by token. At each step t , it uses its previous outputs ($y_{<t}$) and the final N latent vectors (as context C) to predict the next token y_t . The loss is the cross-entropy between the model's predicted probability distribution and the ground-truth next token, summed over the entire report sequence.

(c) **Training Strategy****Pre-training Objective: Multimodal Masked Autoencoding (MultiMAE)**

- **Objective:** From a large dataset of paired X-rays and reports, we take a single pair. We heavily mask a random subset of the image patch tokens (e.g., 75% masked) and lightly mask a random subset of the text tokens (e.g., 15% masked). The model's task is to reconstruct the original content of the masked tokens using the unmasked context from *both* modalities. The reconstruction loss for image tokens is typically Mean Squared Error (MSE) on pixel values, while for text tokens, it is cross-entropy loss.
- **Data Requirement:** This requires a large-scale dataset of image-text pairs, such as the MIMIC-CXR dataset, which contains hundreds of thousands of chest X-rays and their associated free-text reports. The reports do not need to be structured.
- **Benefit:** This pre-training forces the model to learn a deep, fine-grained alignment between visual patterns and their textual descriptions. This builds a robust, foundation, improving performance when fine-tuning on the smaller, specialized structured reporting task.

A CLIP-style contrastive objective would also be an acceptable answer.

(d) **Computational Analysis**

Given: $M = 4352$, $d = 768$, $N = 256$, $L = 12$.

- i. **Cost of Architecture A (Single Encoder Self-Attention Layer)** The bottleneck is self-attention over the full sequence of length M .

$$\text{FLOPs}_A \approx 2 \cdot M^2 \cdot d = 2 \cdot (4352)^2 \cdot 768 \approx 2.90 \times 10^{10} \text{ FLOPs}$$

- ii. **Cost of Architecture B (Encoding Components)** The cost is the sum of the initial cross-attention and the latent processing tower.

$$\begin{aligned}\text{FLOPs}_{\text{B,cross}} &= 2 \cdot N \cdot M \cdot d = 2 \cdot 256 \cdot 4352 \cdot 768 \approx 1.71 \times 10^9 \\ \text{FLOPs}_{\text{B,latent}} &= L \cdot (2 \cdot N^2 \cdot d) = 12 \cdot (2 \cdot 256^2 \cdot 768) \approx 1.20 \times 10^9 \\ \text{FLOPs}_B &= \text{FLOPs}_{\text{B,cross}} + \text{FLOPs}_{\text{B,latent}} \approx 2.91 \times 10^9 \text{ FLOPs}\end{aligned}$$

- iii. **Comparison**

$$\frac{\text{FLOPs}_A}{\text{FLOPs}_B} \approx \frac{2.90 \times 10^{10}}{2.91 \times 10^9} \approx 9.97$$

Architecture A's bottleneck layer is approximately **10 times more computationally expensive** than the entire encoding process of Architecture B.

- iv. **Break-Even Point** We find the input length M_{eq} where the cost of one self-attention layer in A equals the cost of the initial cross-attention layer in B.

$$\begin{aligned}2 \cdot M_{\text{eq}}^2 \cdot d &= 2 \cdot N \cdot M_{\text{eq}} \cdot d \\ M_{\text{eq}}^2 &= N \cdot M_{\text{eq}} \\ M_{\text{eq}} &= N\end{aligned}$$

The result is $M_{\text{eq}} = N = \mathbf{256}$. This value represents the threshold at which the Latent-Query design becomes more efficient. For any input sequence longer than 256 tokens, the quadratic cost of full self-attention grows faster than the linear cost of cross-attention. Since our input length is $M = 4352 \gg 256$, Architecture B offers a substantial computational advantage.

Correction Note: For simplification in a previous version, we omitted the details of the decoding step. For completeness: A decoder generates the output sequence using another cross-attention mechanism. A set of task-specific **output queries** (Q) attend to the final latent array (K/V). These queries are typically learnable embeddings that correspond to the desired output structure (e.g., positional embeddings for an output grid). Each token gets a positional embedding corresponding to its place within its own modality (e.g., a 2D position for an image patch, a 1D position for a text token). The resulting output vectors are then passed to a prediction head to produce the final tokens.